LAPORAN PRAKTIKUM

Abstrac Class and Interface

Dibuat untuk memenuhi salah satu tugas Praktikum Mata Kuliah
Pengembangan Berorientasi Objek



Dibuat Oleh:

NAMA	: Maulida Nur Afifah
NIM	: 251511017
KELAS	: 2-A
PROGRAM STUDI	: D3 Teknik Informatika
JURUSAN	: Teknik Komputer dan Informatika

POLITEKNIK NEGERI BANDUNG

2026

PRAKTIKUM I
EMPLOYEE, MANAGER, AND SORTABLE

Employee.java

```
public class Employee {
    private String name;
    private double salary;
    private int hireDay;
    private int hireMonth;
    private int hireYear;

    public Employee(String name, double salary, int day, int month, int year) {
        this.name = name;
        this.salary = salary;
        this.hireDay = day;
        this.hireMonth = month;
        this.hireYear = year;
    }

    public void print() {
        System.out.println(name + " " + salary + " " + hireYear());
    }

    public void raiseSalary(double byPercent) {
        salary *= 1 + byPercent / 100;
    }

    public int hireYear() {
        return hireYear;
    }

    public double getSalary() {
        return salary;
    }

}
```

Checkpoint 1

State Employee: name, salary, hireDay, hireMonth, hireYear
Behavior Employee: print(), raiseSalary(), hireYear(). getSalary()
Salary tetap dibuat private walaupun Employee akan punya subclass karena subclass tidak boleh mengubah salary langsung dan harus lewat getSalary() juga raiseSalary()

EmployeeTest.java

```
public class EmployeeTest {
    public static void main(String[] args) {
        Employee[] staff = new Employee[3];

        staff[0] = new Employee("Antonio Rossi", 2000000, 1, 10, 1989);
        staff[1] = new Employee("Maria Bianchi", 2500000, 1,12, 1991);
        staff[2] = new Employee("Isabel Vidal", 3000000,1,11,1993);

        for (Employee e : staff) {
            e.raiseSalary(5);
        }

        for (Employee e : staff) {
            e.print();
        }
    }
```

```
}
```

Buat Manager sebagai subclass Employee

Manager.java

```
import java.util.Calendar;
import java.util.GregorianCalendar;

public class Manager extends Employee {
    private String secretaryName;

    public Manager(String name, double salary, int day, int month, int year) {
        super(name, salary, day, month, year);
        secretaryName = "";
    }

    @Override
    public void raiseSalary(double byPercent) {
        GregorianCalendar today = new GregorianCalendar();
        int currentYear = today.get(Calendar.YEAR);
        double bonus = 0.5 * (currentYear - hireYear());
        super.raiseSalary(byPercent + bonus);
    }

    public String getSecretaryName() {
        return secretaryName;
    }
}
```

Checkpoint 2

super(name, salary, day, month, year) dipanggil karena bagian Employee. Semua field name, salary, hireDay, hireMonth, hireYear harus diinisialisasi dulu sebelum Manager bisa berjalan

Uji Employee dan Manager dalam satu program

TestEmployee.java

```
public class EmployeeTest {
    public static void main(String[] args) {
        // Step 2
        System.out.println("=== Step 2: Hanya Employee ===");
        Employee[] onlyEmployee = new Employee[3];
        onlyEmployee[0] = new Employee("Antonio Rossi", 2000000, 1, 10, 1989);
        onlyEmployee[1] = new Employee("Maria Bianchi",  2500000, 1, 12, 1991);
        onlyEmployee[2] = new Employee("Isabel Vidal",   3000000, 1, 11, 1993);

        for (Employee e : onlyEmployee) {
            e.print();
        }
        System.out.println();

        // Step 4
        System.out.println("=== Step 4: Employee + Manager ===");
        Employee[] staff = new Employee[3];
        staff[0] = new Employee("Antonio Rossi", 2000000, 1, 10, 1989);
        staff[1] = new Manager("Maria Bianchi",  2500000, 1, 12, 1991); // Manager
        staff[2] = new Employee("Isabel Vidal",   3000000, 1, 11, 1993);

        System.out.println("Sebelum raiseSalary:");
        for (Employee e : staff) {
            e.print();
        }

        // Setiap object menjalankan raiseSalary() versinya sendiri
        for (Employee e : staff) {
            e.raiseSalary(5);
```

```
        }

        System.out.println("Sesudah raiseSalary(5):");
        for (Employee e : staff) {
            e.print();
        }
        System.out.println();
    }
}
```

```
Filter (e.g. text, !excludeText, text1,text2)

=== Step 4: Employee + Manager ===
Sebelum raiseSalary:
Antonio Rossi 2000000.0 1989
Maria Bianchi 2500000.0 1991
Isabel Vidal 3000000.0 1993
Sesudah raiseSalary(5):
Antonio Rossi 2100000.0 1989
Maria Bianchi 3062500.0 1991
Isabel Vidal 3150000.0 1993
```

Checkpoint 3

Manager adalah sebuah Employee sehingga object Manager dapat ditempatkan pada array Employee[]. Array Employee[] menampung object bertipe Employee atau subclassnya.

Buat abstract class sortable

Sortable.java

```
public abstract class Sortable {
    public abstract int compare(Sortable other);
}
```

```
[Running] cd "d:\POLBAN-2025\smt3\PBO\Praktek\praktikum5\src\" && javac EmployeeTest.java && java EmployeeTest
EmployeeTest.java:38: error: Sortable is abstract; cannot be instantiated
        Sortable s = new Sortable();
                     ^
1 error
```

Checkpoint 4

Abstract class tidak bisa dijadikan object langsung karena ada mthod yang belum punya implementasi

Jadikan Employee subclass dari Sortable

Checkpoint 5

```
[Running] cd "d:\POLBAN-2025\smt3\PBO\Praktek\praktikum5\src\" && javac EmployeeTest.java && java EmployeeTest
.\Employee.java:1: error: Employee is not abstract and does not override abstract method compare(Sortable) in Sortable
public class Employee extends Sortable {
       ^
1 error
```

```java
src > Employee.java > Employee > compare(Sortable)
public class Employee extends Sortable {

    @Override
    public int compare(Sortable other) {
        Employee otherEmployee = (Employee) other;
        if (getSalary() < otherEmployee.getSalary()) return -1;
        if (getSalary() > otherEmployee.getSalary()) return  1;
        return 0;
    }
}
```

Sortable mewajibkan Employee untuk menyediakan implementasi compare(Sortable other)

	Lokasi	Isinya
Di Sortable	Deklarasi aja	public abstract int compare(Sortable other); tidak ada body
Di Employee	Implementasi konkret	Logika membandingkan salary dan return -1/0/1

Uji compare(..)

```java
        // Step 7
        System.out.println(x: "=== Step 7: Uji compare() ===");
        System.out.println("staff[0] vs staff[1]: " + staff[0].compare(staff[1]));
        System.out.println("staff[1] vs staff[2]: " + staff[1].compare(staff[2]));
        System.out.println("staff[2] vs staff[0]: " + staff[2].compare(staff[0]));
        System.out.println();
```

```
oblems 2  Output  Debug Console  Terminal  Ports  Spell Checker 17

=== Step 7: Uji compare() ===
staff[0] vs staff[1]: -1
staff[1] vs staff[2]: -1
staff[2] vs staff[0]: 1
```

```java
public class Manager extends Employee extends Sortable {
    private String secretaryName;

    public Manager(String name, double salary, int day, int month, int year) {
        super(name, salary, day, month, year);
        secretaryName = "";
    }
```

```
Problems 1  Output  Debug Console  Terminal  Ports  Spell Checker 11  Filter (e.g. text, !excludeText, t...

[Running] cd "d:\POLBAN-2025\smt3\PBO\Praktek\praktikum5\src\" && javac EmployeeTest.java && java EmployeeTest
.\Manager.java:4: error: '{' expected
public class Manager extends Employee extends Sortable {
                                               ^
1 error
```

```
45          // Step 8B
46          System.out.println(x: "=== Step 8B: Inheritance Chain Manager ===");
47          Manager  m = new Manager(name: "Maria Bianchi",  salary: 2500000, day: 1, month: 12, year: 1991);
48          Employee e = new Employee(name: "Isabel Vidal",   salary: 3000000, day: 1, month: 11, year: 1993);
49          System.out.println("m.compare(e): " + m.compare(e));
50      }
51  }
```

Problems 2 | Output | Debug Console | Terminal | Ports | Spell Checker 12 | Filter (e.g. text, !excludeText, t...

```
Isabel Vidal 3000000.0 1993
Sesudah raiseSalary(5):
Antonio Rossi 2100000.0 1989
Maria Bianchi 3062500.0 1991
Isabel Vidal 3150000.0 1993

=== Step 7: Uji compare() ===
staff[0] vs staff[1]: -1
staff[1] vs staff[2]: -1
staff[2] vs staff[0]: 1

=== Step 8B: Inheritance Chain Manager ===
m.compare(e): -1
```

Checkpoint 6

Manager tetap punya compare() karena inheritance bersifat transitif. Manager mewarisi semua yang dimiliki Employee termasuk compare)_ yang Employee dapat dari Sortable

Sortable
↑extends
Employee ← punya compare() implementasi konkret
↑extends
Manager ← mewarisi compare() dari Imployee

PRAKTIKUM II
GOODS AND TAXABLE

```
Goods
- description
- price
+ display()

Food extends Goods
- calories

Toy extends Goods implements Taxable
- minimumAge

Book extends Goods implements Taxable
- author

Taxable
- taxRate = 0.06
+ calculateTax()
```

State yang cukup umum sehingga ada di Goods: description dan price
State yang hanya dimiliki Food, Toy, dan Book: calories (Food), minimumAge (Toy), author (Book)
Taxable tidak diterapkan pada semua subclass Goods karena tidak semua class memiliki kontrak dengan Taxable

Buat superclass Goods
Goods.java

```
public class Goods {

    private String description;
    private double price;

    public Goods(String description, double price) {
        this.description = description;
        this.price       = price;
    }

    public String getDescription() { return description; }
    public double getPrice() { return price; }

    public void display() {
        System.out.println("Description : " + description);
        System.out.println("Price       : " + price);
    }
}
```

Checkpoint 7

description dan price diletakkan di Goods karena state yang umum dan ada di setiap barang dan untuk menjaga konsistensi, jadi nanti satu perubahan di Goods otomatis berlaku untuk semua subclassnya

Buat Food sebagai subclass Goods
Food.java

```
public class Food extends Goods {
```

```
    private int calories;

    public Food(String description, double price, int calories) {
        super(description, price);
        this.calories = calories;
    }

    public int getCalories() {
        return calories;
    }

    @Override
    public void display() {
        super.display();
        System.out.println("Calories    : " + calories);
    }
}
```

Buat Interface Taxable

Taxable.java

```
public interface Taxable {
    double taxRate = 0.06;
    double calculateTax();
}
```

Checkpoint 8

Keduanya tidak bisa diinstansiasi langsung, persamaan hasil eksperimen `new Sortable()` dan `new Taxable()` sama-sama menghasilkan error, hanya bisa digunakan sebagai tipe reference

Perbedaan deklarasi:

	Sortable	Taxable
Keyword	Public abstract class	Public interface
bisa punya field instance?	Ya	Tidak, hanya konstanta
Bisa punya method konkret?	ya	Tidak, defaultnya abstract
Digunakan dengan	extends	implements

Buat Toy yang extends Goods dan implements

Toy.java

```
public class Toy extends Goods implements Taxable {
    private int minimumAge;

    public Toy(String description, double price, int minimumAge) {
        super(description, price);
        this.minimumAge = minimumAge;
    }

    public int getMinimumAge() { return minimumAge; }

    @Override
    public double calculateTax() {
        return getPrice() * taxRate;
    }

    @Override
    public void display() {
        super.display();
        System.out.println("Minimum age : " + minimumAge);
        System.out.println("Tax         : " + calculateTax());
    }
}
```

Checkpoint 9

`public class Toy extends Goods implements Taxable`

extends Goods menyatakan ”Toy adalah Goods” (is-a relationship inheritance)
implements Taxable menyatakan “Toy memenuhi kontrak Taxable” (has-a contract interface)

Buat Book yang extends Goods dan implements Taxable

Book.java

```
public class Book extends Goods implements Taxable {
    private String author;

    public Book(String description, double price, String author) {
        super(description, price);
        this.author = author;
    }

    public String getAuthor() {
        return author;
    }

    @Override
    public double calculateTax() {
        return getPrice() * taxRate;
    }

    @Override
    public void display() {
        super.display();
        System.out.println("Author : " + author);
        System.out.println("Tax    : " + calculateTax());
    }
}
```

Uji hierary dan constract secara utuh

GoodsTest.java

```
public class GoodsTest {
    public static void main(String[] args) {
        Food food = new Food("Bread", 15000, 250);
        Toy toy = new Toy("Robot", 120000, 8);
        Book book = new Book("Java Basics", 90000, "A. Programmer");

        food.display();
        System.out.println();
        toy.display();
        System.out.println();
        book.display();

        System.out.println();
        Taxable taxableToy = toy;
        Taxable taxableBook = book;
        System.out.println("Toy tax  : " + taxableToy.calculateTax());
        System.out.println("Book tax : " + taxableBook.calculateTax());
    }
}
```

```
[Running] cd "d:\POLBAN-2025\smt3\PBO\Praktek\praktikum5\src\" && javac GoodsTest.java && java GoodsTest
Description : Bread
Price       : 15000.0
Calories    : 250

Description : Robot
Price       : 120000.0
Minimum age : 8
Tax         : 7200.0

Description : Java Basics
Price       : 90000.0
Author      : A. Programmer
Tax         : 5400.0

Toy tax  : 7200.0
Book tax : 5400.0

[Done] exited with code=0 in 0.564 seconds
```

Sengaja langar kontrak interface

```
[Running] cd "d:\POLBAN-2025\smt3\PBO\Praktek\praktikum5\src\" && javac GoodsTest.java && java GoodsTest
.\Toy.java:1: error: Toy is not abstract and does not override abstract method calculateTax() in Taxable
public class Toy extends Goods implements Taxable {
       ^
.\Toy.java:22: error: cannot find symbol
        System.out.println("Tax         : " + calculateTax());
                                              ^
  symbol:   method calculateTax()
  location: class Toy
2 errors
```

Checkpoint 10

Keyword implements bukan sekadar label. Ketika sebuah class implements interface, compiler menuntut class konkret tersebut memenuhi method yang dideklarasikan interface.

Tabel perbandingan abstract class dan interface

Pertanyaan	Sortable (abstract class)	Taxable (Interface)
Dapat dibuat object langsung?	Tidak	Tidak
Menggunakan extends atau implements?	extends	implements
Dapat mempunyai implementasi method konkret?	Iya, bisa punya method biasa selain method abstract	Tidak, semua method defaultnya abstract
Peran utamanya pada kasus praktikum?	Mendefiniskan kewajiban compare() untuk semua Employee yang bisa dibandingkan dan Employee wajib punya bentuk konkretnya	Menetapkan kontrak calculateTax() untuk class yang ingin diatur untu kena pajak tanpa memaksa mereka punya parent class baru
Apakah class dapat mempunyai parent class lain sekaligus?	Tidak, hanya satu parent class saja	Iya, Toy bisa extends Goods sekaligus implements taxable

Refleksi

1. Mengapa Sortable berguna walaupun object Sortable tidak pernah dibuat?

Karena Sortable mendefinisikan kewajiban, siapapun yang extends Sortable dijamin punya compare()

2. Apa yang terjadi ketika Employee extends Sortable tetapi belum membuat compare()?

Java menolak class konkret yang methodnya belum dibuat

3. Mengapa Manager tetap dapat menggunakan compare() tanpa menulis extends Sortable?

Karena inheritance bersifat transitif. Dengan Manager extends Employee, Employee extends Sortable maka Manager otomatis mewarisi semua yang Employee punya termasuk compare()

4. Mengapa `class Manager extends Employee extends Sortable` tidak diperlukan dan tidak valid di Java?

Tidak diperlukan karena Manager sudah mewarisi Sortable. Tidak valid karena java menerapkan single inheritance

5. Apa yang dijanjikan Toy ketika menulis `implements Taxable`?

Toy berjanji akan pasti menyediakan calculateTax() yang bisa dipanggil siapapun. Kalau Toy tidak menepati janjinya maka program tidak akan bisa dicompile

6. Mengapa Goods lebih tepat menjadi superclass bagi Food/Toy/Book, sedangkan Taxable lebih tepat menjadi interface pada model ini?

Goods tepat menjadi superclass karena Food, Toy, dan Book benar benar is-a Goods. Mereka berbagi state dan behavior yang sama.
Taxable tepat menjadi interface karena tidak semua Foods dikenakan pajak

7. Apakah bukti dari compiler bahwa interface benar-benar merupakan contract?

Saat calculateTax() dihapus dari Toy yang masih implements Taxable, compiler error:

```
error: Toy is not abstract and does not override
       abstract method calculateTax() in Taxable
```

Compiler tidak bisa membiarkan Toy mengaku implements Taxable tanpa benar benar menyediakan methodnya.

8. Lengkapi dengan bahasa Anda sendiri: "Abstract class membantu saya ketika ..., sedangkan interface membantu saya ketika ..."

Abstract class membantu saya ketika class-class dalam satu hierarchy punya kesamaan umum dan ada behavior yang wajib ada tapi cara implementasinya beda di setiap subclass, sedangkan interface membantu saya ketika kontrak/kemampuan perlu diberikan ke beberapa class yang mungkin tidak berada dalam hierarchy yang sama